

Python Closures & nonlocal

Student Handout

A simple guide with examples, mental models, and practice

1. What is a Closure?

A closure happens when an inner function remembers a variable from the outer function, even after the outer function has finished running.

Simple idea:

Outer function → creates a value → inner function remembers that value → returned function can use it later.

2. Basic Closure Example

```
def greet(name):  
    def say_hello():  
        return f"Hello, {name}!"  
    return say_hello
```

```
hello = greet("Ali")  
print(hello()) # Hello, Ali!
```

Here, `say_hello()` remembers the name "Ali". The variable name belongs to `greet()`, but the returned function can still use it later.

3. Reading an Outer Variable

An inner function can normally READ a variable from an enclosing function without using `nonlocal`.

```
def make_multiplier(factor):  
    def multiply(n):  
        return n * factor  
  
    return multiply
```

```
double = make_multiplier(2)  
triple = make_multiplier(3)
```

```
print(double(5)) # 10  
print(triple(5)) # 15
```

`double` remembers `factor = 2`, while `triple` remembers `factor = 3`. They have separate closure state.

4. Why nonlocal is Needed

If an inner function ASSIGNS to a name, Python normally treats that name as local to the inner function.

```
def make_counter():
    count = 0

    def increment():
        count += 1
        return count

    return increment
```

This causes an error because Python sees `count += 1` as an assignment to a local variable, but that local count has not been given a value.

5. nonlocal: Modify the Enclosing Variable

`nonlocal` tells Python: "Use the variable from the nearest enclosing function instead of creating a new local variable."

```
def make_counter():
    count = 0

    def increment():
        nonlocal count
        count += 1
        return count

    return increment

counter = make_counter()

print(counter()) # 1
print(counter()) # 2
print(counter()) # 3
```

The same count is updated on every call. The closure remembers it.

6. Closure vs nonlocal

Concept	What it does	Example
Closure	Lets an inner function remember outer variables.	<code>make_multiplier()</code>
nonlocal	Lets an inner function modify an enclosing variable.	<code>make_counter()</code>
local	Variable belongs to the current function.	<code>x = 10</code> inside a function
global	Variable belongs to the module/global scope.	<code>global x</code>

Important: nonlocal does NOT create the closure. The nested function's reference to an enclosing variable is what allows the closure; nonlocal is specifically about modifying that enclosing variable.

7. Another Example: Power Function

```
def make_power(exponent):
    def power(number):
        return number ** exponent
    return power
```

```
square = make_power(2)
cube = make_power(3)
```

```
print(square(4)) # 16
print(cube(4))   # 64
```

square remembers exponent = 2. cube remembers exponent = 3.

8. Another Example: Greeting Customizer

```
def make_greeting(prefix):
    def greet(name):
        return f"{prefix}, {name}!"
    return greet
```

```
say_hi = make_greeting("Hi")
say_welcome = make_greeting("Welcome")
```

```
print(say_hi("Sara"))    # Hi, Sara!
print(say_welcome("Ahmed")) # Welcome, Ahmed!
```

9. Closure with Multiple Outer Variables

```
def make_profile(name, age):
    def show_profile():
        return f"{name} is {age} years old."
    return show_profile
```

```
profile = make_profile("Ayesha", 21)
print(profile()) # Ayesha is 21 years old.
```

A closure can remember more than one variable from its enclosing scope.

10. Another nonlocal Example: Accumulator

```
def make_accumulator():
    total = 0
```

```

def add(amount):
    nonlocal total
    total += amount
    return total

return add

total = make_accumulator()

print(total(10)) # 10
print(total(5)) # 15
print(total(20)) # 35

```

The accumulator remembers total and keeps updating it.

11. Multiple Closures Have Separate State

```

counter_a = make_counter()
counter_b = make_counter()

print(counter_a()) # 1
print(counter_a()) # 2

print(counter_b()) # 1
print(counter_b()) # 2

```

counter_a and counter_b were created by separate calls to make_counter(), so each has its own count.

12. The Scope Rule to Remember

When Python looks for a variable, a useful simplified model is LEGB:

- L — Local: current function
- E — Enclosing: outer/nested function
- G — Global: module-level variable
- B — Built-in: Python's built-in names such as print, len, and sum

13. Common Mistakes

Mistake: resetting the variable inside the inner function. count = 0 inside increment() makes it start from zero on every call.

Mistake: forgetting nonlocal when modifying an enclosing variable. Use nonlocal count before count += 1.

Mistake: thinking nonlocal means global. nonlocal refers to an enclosing function's variable, not a global variable.

Mistake: thinking every nested function automatically needs nonlocal. No. If the inner function only reads the outer variable, nonlocal is unnecessary.

14. Quick Comparison

```
# Reading → no nonlocal
def make_multiplier(factor):
    def multiply(n):
        return n * factor
    return multiply

# Modifying → nonlocal
def make_counter():
    count = 0
    def increment():
        nonlocal count
        count += 1
    return count
return increment
```

15. When Are Closures Useful?

- Creating customized functions.
- Keeping private state without using a class.
- Counters, accumulators, and trackers.
- Callbacks and event handlers.
- Configuration functions where settings are remembered.
- Decorators and other advanced Python patterns.

16. Practice Questions

1. Create `make_multiplier(factor)` that returns a function multiplying a number by factor.
2. Create `make_power(exponent)` that returns a function raising a number to exponent.
3. Create `make_counter()` that starts at 100 and increases by 1 on every call.
4. Create `make_accumulator()` that remembers a running total.
5. Create `make_discount(rate)` that returns a function applying the remembered discount rate.
6. Explain why `nonlocal` is needed in a counter but not in a simple multiplier.

17. One-Minute Summary

- Closure = an inner function remembers values from an outer function.
- Reading an enclosing variable usually needs no special keyword.
- Assignment inside the inner function normally creates a local variable.
- `nonlocal` = modify a variable belonging to an enclosing function.
- `global` = modify a variable belonging to the global/module scope.
- Separate calls to the outer function create separate closure state.

Mental model: Closure = remember. nonlocal = modify what you remembered.

18. Example: Building a Tasbeeh Counter

A Tasbeeh counter is a simple real-world example of maintaining state. Closures are one way to do it, but they are not required.

A. Without Closures: Global Variable

```
count = 0
```

```
def tasbeeh():  
    global count  
    count += 1  
    return count
```

```
print(tasbeeh()) # 1  
print(tasbeeh()) # 2  
print(tasbeeh()) # 3
```

This works, but the count is stored in the global scope. In larger programs, global state can become difficult to manage.

B. Without Closures: Using a Class

```
class Tasbeeh:  
    def __init__(self):  
        self.count = 0  
  
    def increment(self):  
        self.count += 1  
        return self.count  
  
    def reset(self):  
        self.count = 0
```

```
tasbeeh = Tasbeeh()
```

```
print(tasbeeh.increment()) # 1  
print(tasbeeh.increment()) # 2  
print(tasbeeh.increment()) # 3
```

```
tasbeeh.reset()  
print(tasbeeh.increment()) # 1
```

Here, `self.count` stores the state inside the `Tasbeeh` object. A class is often easier to extend with features such as `reset()`, a target count, history, or other settings.

C. With a Closure

```
def make_tasbeeh():  
    count = 0  
  
    def increment():  
        nonlocal count  
        count += 1  
        return count  
  
    return increment  
  
tasbeeh = make_tasbeeh()  
  
print(tasbeeh()) # 1  
print(tasbeeh()) # 2  
print(tasbeeh()) # 3
```

The closure remembers count, while nonlocal allows increment() to modify that remembered variable.

D. Comparing the Three Approaches

Approach	Where the count lives	Key idea
Global variable	Global/module scope	Simple, but global state can be hard to manage.
Class	Inside an object as self.count	Good for multiple methods and larger stateful programs.
Closure	Enclosing function scope	Good for small private state and function factories.

19. Key Takeaway

A closure is not the only way to keep state. You can use a global variable, a class, or a closure. The closure approach is especially useful when a function needs to remember some private data between calls. The class approach becomes attractive when you need several related operations on that state.